PRIVACY-PRESERVING DATA ANALYSIS

Lecture 1 - Reconstruction attack

Static study notes. The interactive version, with live demos and settings, is on the website.

A table you can only query

Picture a hospital table. Each row is a patient. One column is secret: **sick (1)** or **healthy (0)**. Nobody may read it directly.

You may only ask about **groups**: “how many sick people are in this batch?” A batch can be one patient, everyone, or anything in between.

To protect patients, the server adds random noise to every answer. The noise is never bigger than a fixed limit, **E = α × n**.

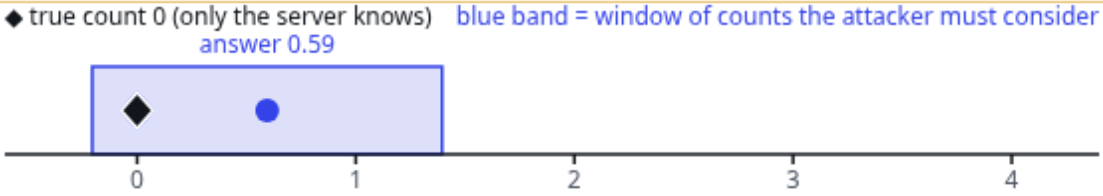
> **WATCH FOR**
> - The demo asks about one patient, then everyone, then four patients.
> - The answers are not whole numbers: that's the noise.

`n`	number of patients (rows)
`batch`	any group of rows you ask about
`α`	how strong the noise is, as a share of n
`E = αn`	the largest error an answer can have

The hospital table click rows, then ask your own batch

#	PATIENT	AGE	ZIP	SECRET: SICK?
1	Patient 01	50	14060	0
2	Patient 02	30	17450	0
3	Patient 03	42	15020	0
4	Patient 04	78	16040	0
5	Patient 05	75	19790	0
6	Patient 06	52	19650	0
7	Patient 07	53	15490	0
8	Patient 08	42	16110	1

Query 3: how many sick among 4 patients? The true count is **0**, but the server answered **0.59** (noise +0.59). The attacker only knows the truth is within 0.80 of that, so it could be **0, 1**.
Your turn: click rows and press **Ask selected batch**.



Last batches asked

QUERY	BATCH (DARK = INCLUDED)	PEOPLE	TRUE COUNT	ANSWER	NOISE
#3 yours	■■■■	4	0	0.59	+0.59
#2 yours	■■■■■■■■	8	1	1.32	+0.32
#1 yours	■	1	0	-0.09	-0.09

Queries asked: 3 of 255 possible batches

One noisy answer is a window, not a number

The attacker gets the server's answer, say 2.6. They know the noise is at most **E**, so the true count must be within E of 2.6. That is a **window** (the blue band below the table).

Here the noise is stronger (α = 0.25, so E = 2). The window is wide enough to hold several whole numbers, so **one answer barely narrows things down**. Taken alone, this looks private.

WATCH FOR

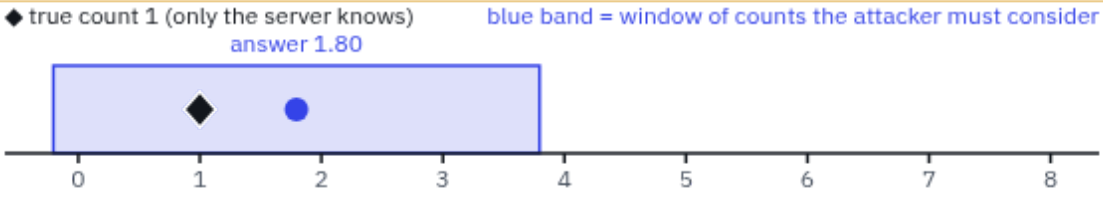
- The diamond is the real count. It always sits inside the band.
- Try asking the same batch twice: you get the same answer. If the server re-rolled its noise, the attacker could average it away.

The hospital table

click rows, then ask your own batch

#	PATIENT	AGE	ZIP	SECRET: SICK?
1	Patient 01	50	14060	0
2	Patient 02	30	17450	0
3	Patient 03	42	15020	0
4	Patient 04	78	16040	0
5	Patient 05	75	19790	0
6	Patient 06	52	19650	0
7	Patient 07	53	15490	0
8	Patient 08	42	16110	1

Query 3: how many sick among everyone? The true count is **1**, but the server answered **1.80** (noise +0.80). The attacker only knows the truth is within 2.00 of that, so it could be **0, 1, 2, 3**.



Last batches asked

QUERY	BATCH (DARK = INCLUDED)	PEOPLE	TRUE COUNT	ANSWER	NOISE
#3 yours	■■■■■■■■	8	1	1.80	+0.80
#2 yours	▫▫■■■■■■	6	1	-0.40	-1.40
#1 yours	■■■■■■▫▫	6	0	0.78	+0.78

Queries asked: 3 of 255 possible batches

Ask everything, cross out the impossible

But the attacker is not limited to a few questions. With n rows there are only 2^n-1 batches, so they **ask all of them**.

They also write down every possible secret column: 2^n **candidates** (the grid). Each answer crosses out every candidate whose count for that batch falls outside the window. The real column can never be crossed out, because the noise really is at most E.

As candidates disappear, the black bars in the table peel away: the attacker is getting sure of each patient.

WATCH FOR

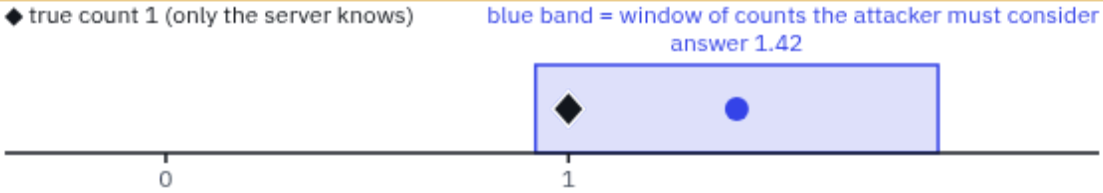
- Green cells are survivors. Blue flashes are cells just eliminated.
- With small noise ($\alpha = 0.05$) only **one** candidate survives: the whole secret column has leaked.

The hospital table click rows, then ask your own batch

#	PATIENT	AGE	ZIP	SECRET: SICK?	ATTACKER'S RECONSTRUCTION	MATCH
1	Patient 01	53	12550	0	0	✓ match
2	Patient 02	18	19010	1	1	✓ match
3	Patient 03	58	19870	0	0	✓ match
4	Patient 04	36	14980	1	1	✓ match
5	Patient 05	41	13470	0	0	✓ match
6	Patient 06	45	12360	0	0	✓ match
7	Patient 07	40	15280	1	1	✓ match
8	Patient 08	73	14500	0	0	✓ match
9	Patient 09	23	18800	0	0	✓ match
10	Patient 10	58	18830	0	0	✓ match

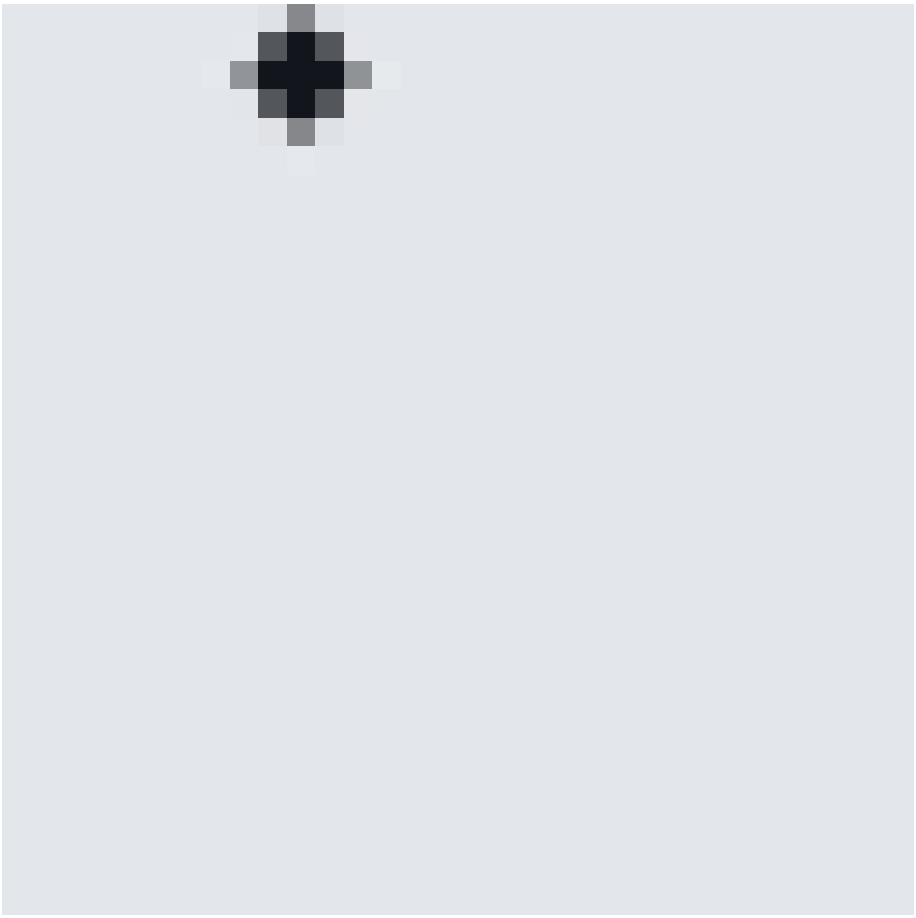
Query 1023: how many sick among one patient? The true count is **1**, but the server answered **1.42** (noise +0.42). The attacker only knows the truth is within 0.50 of that, so it could be **1**. Every candidate column whose count for this batch falls outside that window is crossed out: **1** → **1** candidates left.

All batches asked. Majority vote gets **0** of 10 bits wrong (0%); the worst survivor is 0 bits off against a guarantee of 2.0.



Every possible secret column

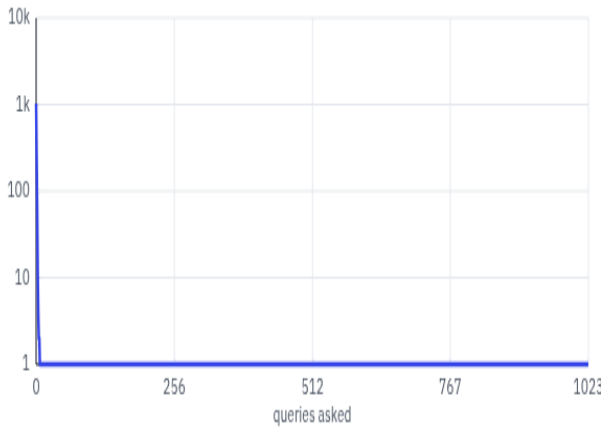
1 of 1,024 still alive. The diamond marks the true column.



survivor, 0 bits wrong
survivor, some bits wrong
survivor, many bits wrong
just eliminated
eliminated

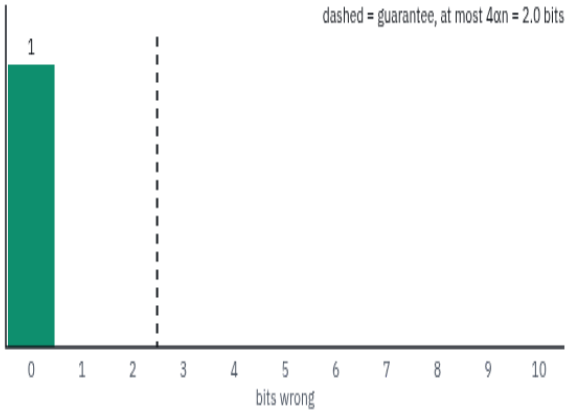
Candidates still alive

log scale



How wrong are the survivors?

bits wrong, and how many survivors



NOISE BOUND E = AN

0.50

every answer is off by at most this

GUARANTEE

2.0 bits

any survivor is at most 4αn wrong

SURVIVORS

1

the true column is among them

ATTACKER OUTPUT (MAJORITY VOTE)

0 wrong

0% error rate

ONE RANDOM SURVIVOR

0 wrong

0% error rate

BEST / WORST SURVIVOR

0 / 0

worst is within 4αn, as the theorem says

Last batches asked

QUERY	BATCH (DARK = INCLUDED)	PEOPLE	TRUE COUNT	ANSWER	NOISE	ALIVE AFTER
#1023		1	1	1.42	+0.42	1 (−0)
#1022		6	2	2.18	+0.18	1 (−0)
#1021		5	0	-0.41	-0.41	1 (−0)
#1020		5	1	0.99	-0.01	1 (−0)
#1019		4	1	0.91	-0.09	1 (−0)
#1018		5	1	1.14	+0.14	1 (−0)
#1017		3	1	1.07	+0.07	1 (−0)
#1016		4	1	1.07	+0.07	1 (−0)

Queries asked: 1,023 of 1,023 possible batches

When noise is big: the 4αn guarantee

Now turn the noise up (α = 0.2). The server also gets smarter: it answers as if the table were **halfway between the truth and a fake table** that differs in some rows. Both the real table and the fake one fit every window, so neither can be crossed out.

Theory says that **any survivor is at most 4αn bits wrong**. Why: take a survivor and the truth. Where they disagree in one direction (a rows), ask about exactly those rows. Both are within E of the same answer, so they can differ by at most 2E there. The same holds for the other direction. Total: 4E = 4αn.

WATCH FOR

- The histogram: bars are survivors grouped by bits wrong. The dashed line is the guarantee; nothing sits past it.
- The table still gets mostly reconstructed: majority vote over survivors is far better than the worst case.

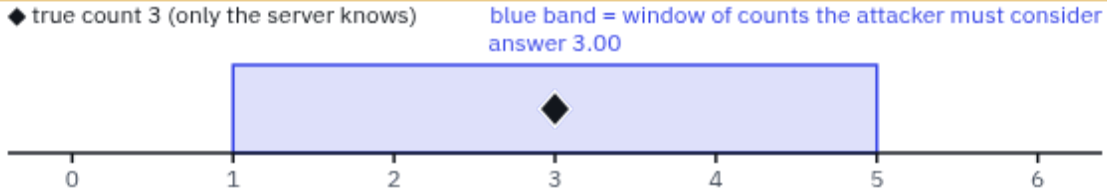
The hospital table

click rows, then ask your own batch

#	PATIENT	AGE	ZIP	SECRET: SICK?	ATTACKER'S RECONSTRUCTION	MATCH
1	Patient 01	53	12550	0	0	✓ match
2	Patient 02	18	19010	1	0	hidden
3	Patient 03	58	19870	0	0	✓ match
4	Patient 04	36	14980	1	1	hidden
5	Patient 05	41	13470	0	0	hidden
6	Patient 06	45	12360	0	0	hidden
7	Patient 07	40	15280	1	1	hidden
8	Patient 08	73	14500	0	0	✓ match
9	Patient 09	23	18800	0	0	✓ match
10	Patient 10	58	18830	0	0	hidden

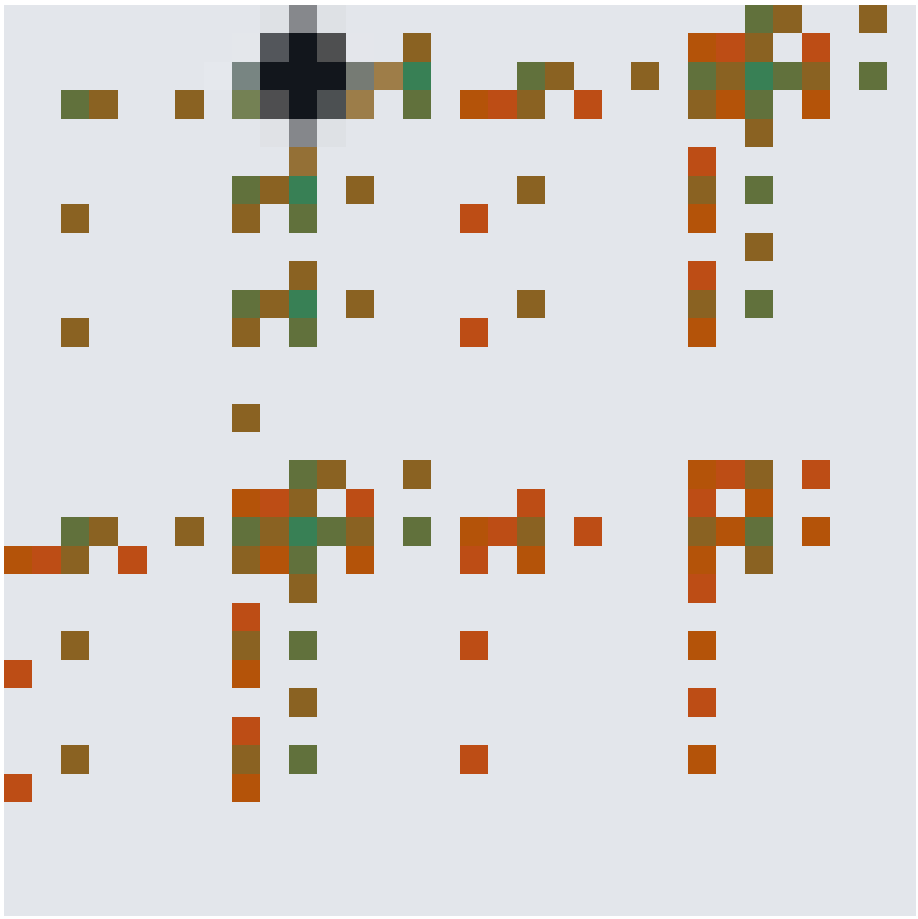
Query 1023: how many sick among 6 patients? The true count is **3**, but the server answered **3.00** (noise +0.00). The attacker only knows the truth is within 2.00 of that, so it could be **1, 2, 3, 4, 5**. Every candidate column whose count for this batch falls outside that window is crossed out: **137 → 137** candidates left.

All batches asked. Majority vote gets **1** of 10 bits wrong (10%); the worst survivor is 5 bits off against a guarantee of 8.0.



Every possible secret column

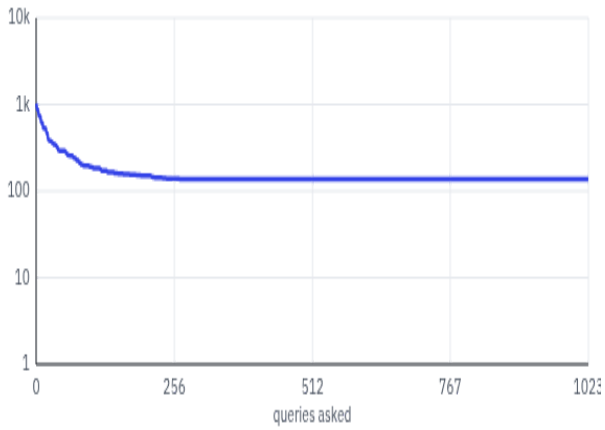
137 of 1,024 still alive. The diamond marks the true column.



survivor, 0 bits wrong
survivor, some bits wrong
survivor, many bits wrong
just eliminated
eliminated

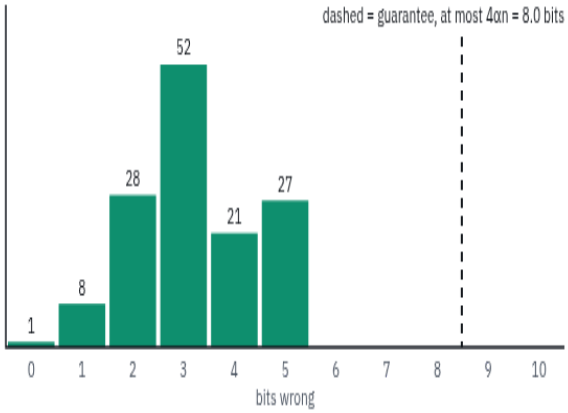
Candidates still alive

log scale



How wrong are the survivors?

bits wrong, and how many survivors



NOISE BOUND E = AN

2.00

every answer is off by at most this

GUARANTEE

8.0 bits

any survivor is at most 4αn wrong

SURVIVORS

137

the true column is among them

ATTACKER OUTPUT (MAJORITY VOTE)

1 wrong

10% error rate

ONE RANDOM SURVIVOR

5 wrong

50% error rate

BEST / WORST SURVIVOR

0 / 5

worst is within 4αn, as the theorem says

Last batches asked

QUERY	BATCH (DARK = INCLUDED)	PEOPLE	TRUE COUNT	ANSWER	NOISE	ALIVE AFTER
#1023		6	3	3.00	+0.00	137 (−0)
#1022		7	2	2.50	+0.50	137 (−0)
#1021		4	2	1.50	-0.50	137 (−0)
#1020		4	2	2.50	+0.50	137 (−0)
#1019		5	2	3.00	+1.00	137 (−0)
#1018		3	1	0.50	-0.50	137 (−0)
#1017		4	2	2.00	+0.00	137 (−0)
#1016		7	2	2.50	+0.50	137 (−0)

Queries asked: 1,023 of 1,023 possible batches

The 4αn proof, on real data

You saw that every survivor is at most 4αn bits wrong. The panel at the top of the stage checks the argument on the actual survivors of this run.

The idea. Take the truth d and any survivor c. Let **A** be the rows where c says sick but the truth says healthy, and **B** the rows where c says healthy but the truth says sick. The attacker asked every batch, so they asked batch A and batch B.

On batch A the truth counts 0 and c counts |A|. Both are within E of the same server answer, so |A| ≤ 2E. Same for B. Total error |A| + |B| ≤ 4E = 4αn.

WATCH FOR

- Press “Show another survivor”. Every one satisfies the same two inequalities.
- The proof only uses “the noise is at most E” and “all 2^n batches were asked”. It never looks at how the noise was chosen, so it holds for any noise.
- The real error is usually well below 4αn; the bound is the worst case.

Proof by example the worst survivor

STEP 1. TRUTH AND A SURVIVOR

truth d	0	1	0	1	0	0	1	0	0	0
survivor c	1	0	0	1	1	1	0	0	0	0

c fits every answer to within E, just like the truth. They differ in **5** rows (outlined in red).

STEP 2. ROWS WHERE C WRONGLY SAYS "SICK"

A = rows **1, 5, 6** (c says sick (1), truth says healthy (0)). The attacker asked this batch, because they asked every batch. The truth's count on A is **0**; c's count is **3**. The server answered **1.00**. The truth is within E = 2.00 of that answer, and so is c. So the two counts differ by at most 2E = **4.00**.

Check: $|A| = 3 \leq 4.00$ ✓

STEP 3. ROWS WHERE C WRONGLY SAYS "HEALTHY"

B = rows **2, 7** (c says healthy (0), truth says sick (1)). The attacker asked this batch, because they asked every batch. The truth's count on B is **2**; c's count is **0**. The server answered **1.50**. The truth is within E = 2.00 of that answer, and so is c. So the two counts differ by at most 2E = **4.00**.

Check: $|B| = 2 \leq 4.00$ ✓

STEP 4. ADD THEM UP

Bits wrong = $|A| + |B| = 3 + 2 = \mathbf{5} \leq 2E + 2E = 4E = 4\alpha n = \mathbf{8.00}$ ✓

The hospital table

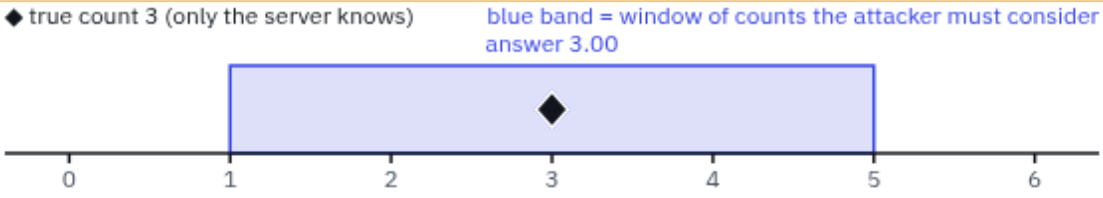
click rows, then ask your own batch

#	PATIENT	AGE	ZIP	SECRET: SICK?	ATTACKER'S RECONSTRUCTION	MATCH
1	Patient 01	53	12550	0	0	✓ match
2	Patient 02	18	19010	1	0	hidden
3	Patient 03	58	19870	0	0	✓ match
4	Patient 04	36	14980	1	1	hidden
5	Patient 05	41	13470	0	0	hidden
6	Patient 06	45	12360	0	0	hidden
7	Patient 07	40	15280	1	1	hidden
8	Patient 08	73	14500	0	0	✓ match
9	Patient 09	23	18800	0	0	✓ match
10	Patient 10	58	18830	0	0	hidden

Query 1023: how many sick among 6 patients? The true count is **3**, but the server answered **3.00** (noise +0.00). The attacker only knows the truth is within 2.00 of that, so it could be **1, 2, 3, 4, 5**. Every candidate column whose count for this batch falls outside that window is crossed out: **137 → 137** candidates left.

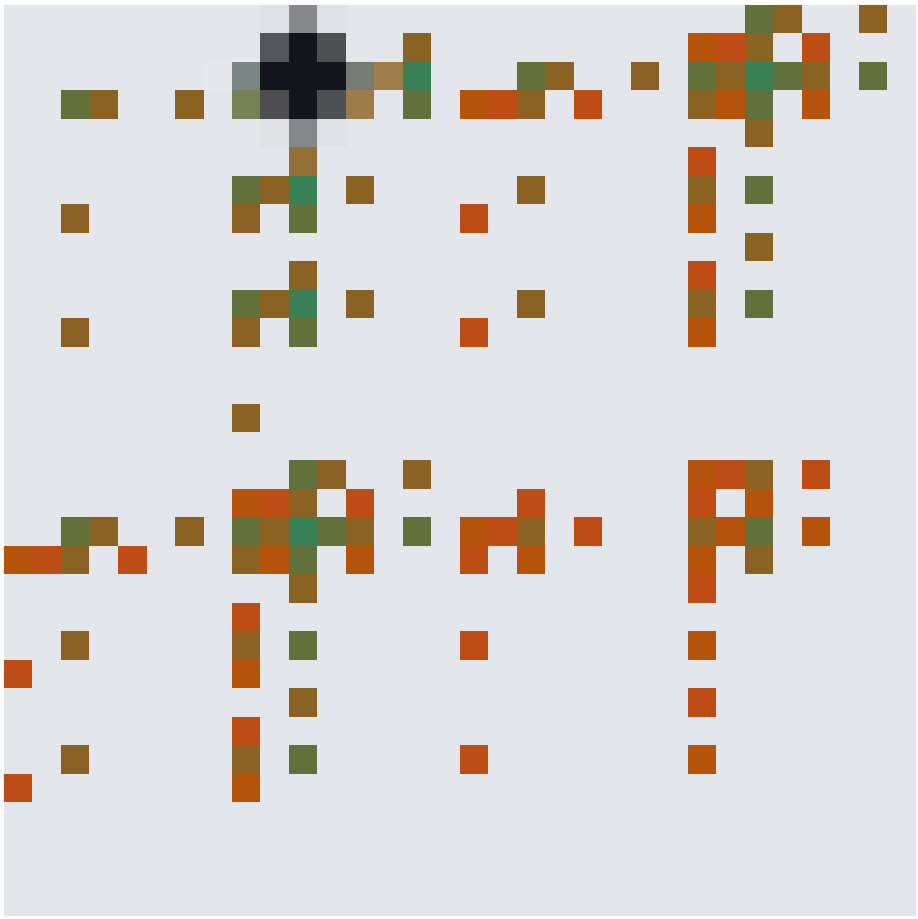
All batches asked. Majority vote gets **1** of 10 bits wrong (10%); the worst survivor is 5 bits off against a guarantee of 8.0.

All batches were asked instantly. Read the proof at the top of the stage.



Every possible secret column

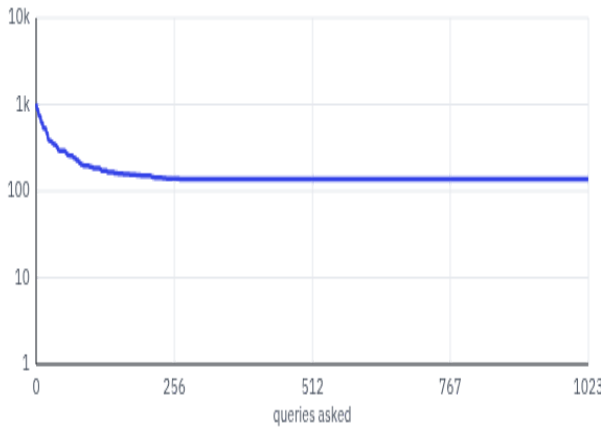
137 of 1,024 still alive. The diamond marks the true column.



survivor, 0 bits wrong
survivor, some bits wrong
survivor, many bits wrong
just eliminated
eliminated

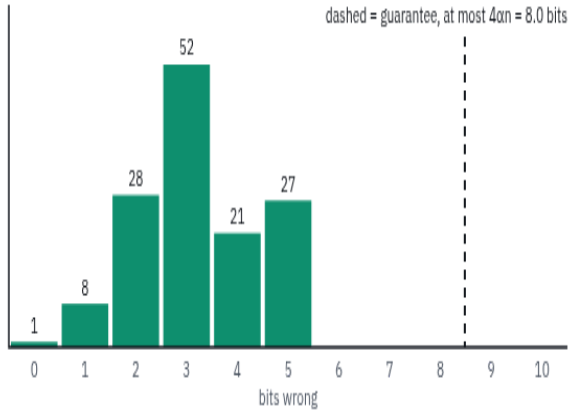
Candidates still alive

log scale



How wrong are the survivors?

bits wrong, and how many survivors



NOISE BOUND E = AN

2.00

every answer is off by at most this

GUARANTEE

8.0 bits

any survivor is at most 4αn wrong

SURVIVORS

137

the true column is among them

ATTACKER OUTPUT (MAJORITY VOTE)

1 wrong

10% error rate

ONE RANDOM SURVIVOR

5 wrong

50% error rate

BEST / WORST SURVIVOR

0 / 5

worst is within 4αn, as the theorem says

Last batches asked

QUERY	BATCH (DARK = INCLUDED)	PEOPLE	TRUE COUNT	ANSWER	NOISE	ALIVE AFTER
#1023		6	3	3.00	+0.00	137 (−0)
#1022		7	2	2.50	+0.50	137 (−0)
#1021		4	2	1.50	-0.50	137 (−0)
#1020		4	2	2.50	+0.50	137 (−0)
#1019		5	2	3.00	+1.00	137 (−0)
#1018		3	1	0.50	-0.50	137 (−0)
#1017		4	2	2.00	+0.00	137 (−0)
#1016		7	2	2.50	+0.50	137 (−0)

Queries asked: 1,023 of 1,023 possible batches

How much noise actually hides the column?

The page now repeats the whole attack for many noise levels α and averages the results.

Right chart. Independent random noise (uniform, bell, Laplace, ±E) is weak against this attacker: errors average out over thousands of questions, so the column is recovered even at high α. Only the **decoy** line rises, because decoy noise is built to be consistent.

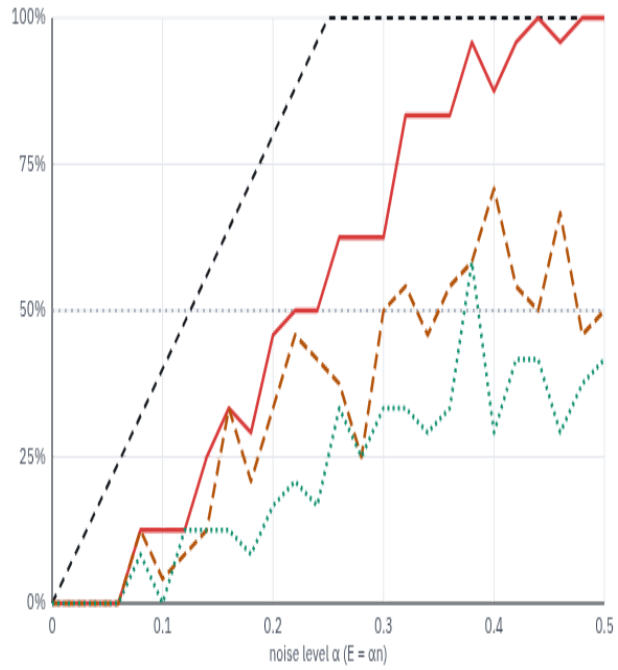
Left chart. One noise type, three ways to read out an answer. The black dashed line is the guarantee 4α. Real errors stay well under it.

WATCH FOR

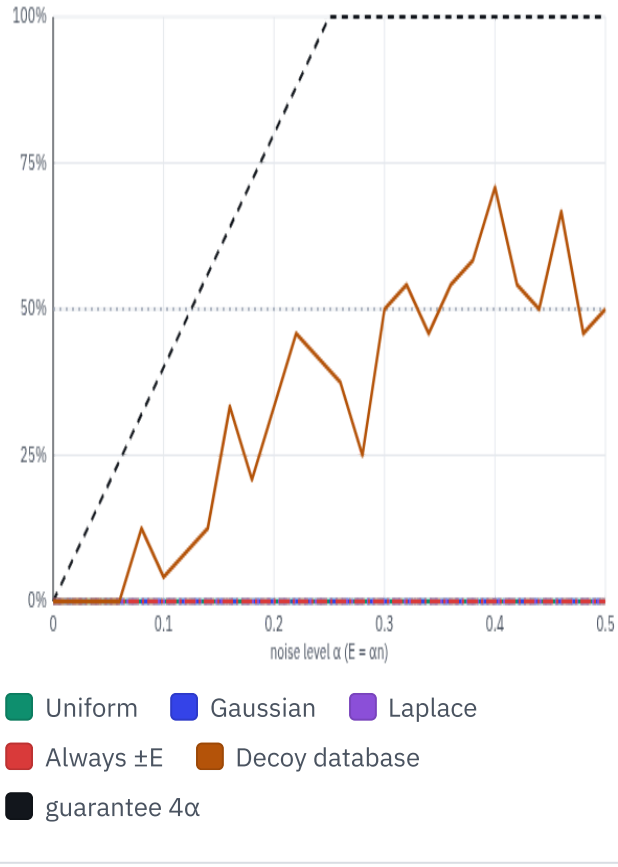
- Every curve stays below the dashed 4α line.
- 50% is a coin flip: the attacker knows nothing.

A · One noise type

Decoy database



B · Random survivor, every noise type



What the noise looks like

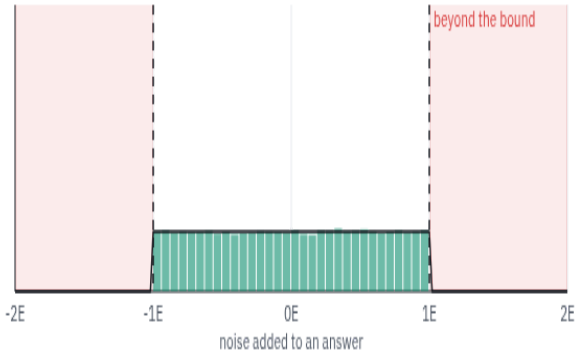
Four noise shapes, all scaled to the same limit E. The dashed lines mark ±E; red shading is beyond the limit.

Bell-shaped and Laplace noise have tails: now and then a value lands beyond E. With thousands of answers, “now and then” becomes “for sure”, so the true column gets crossed out and the clean 4αn guarantee no longer applies. Real privacy systems live in this regime: their noise is not capped, so they control **how many questions** are answered.

WATCH FOR

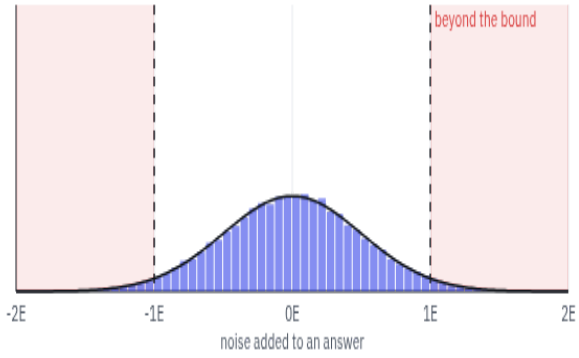
- As E grows, the share of the noise beyond the dashed lines stays the same, but the cards show the chance that **all** answers stay inside.

Uniform flat between −E and +E



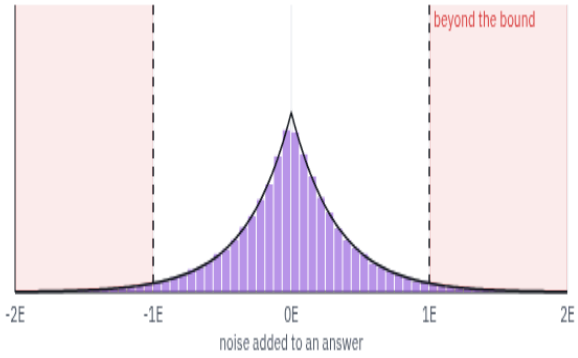
SPREAD (STD DEV)	BEYOND ±E	TRUE COLUMN SURVIVES
4.62	0.000%	100%
√n = 3.16	per answer	all 1,023 answers must stay inside

Gaussian bell curve, σ = E/2



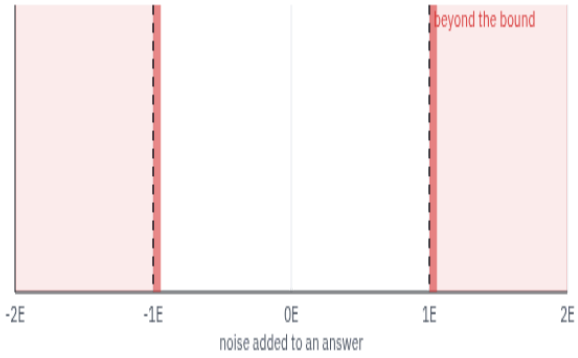
SPREAD (STD DEV)	BEYOND ±E	TRUE COLUMN SURVIVES
4.00	4.6%	0.00%
√n = 3.16	per answer	all 1,023 answers must stay inside

Laplace scale E/3, the classic privacy noise



SPREAD (STD DEV)	BEYOND ±E	TRUE COLUMN SURVIVES
3.77	**5.0%**	**0.00%**
√n = 3.16	per answer	all 1,023 answers must stay inside

Always ±E exactly +E or −E, at random



SPREAD (STD DEV)	BEYOND ±E	TRUE COLUMN SURVIVES
8.00	**0.000%**	**100%**
√n = 3.16	per answer	all 1,023 answers must stay inside

How the √n attack works, step by step

The exponential attack needs 2^n queries. The realistic attack needs only about n·polylog(n) **random** ones. Three steps:

1. **Ask m random batches.** Each patient joins each batch with probability ½.
2. **Find a column that fits every answer.** Any 0/1 vector c with $|\Sigma_S c - \text{answer}(S)| \le E$ for all asked batches. The truth is one. Dinur–Nissim solve this as a linear program over $c \in [0,1]^n$. This page uses least squares (best fit), which behaves similarly and is simpler to run.
3. **Round.** Values of ½ or more become 1, the rest 0.

Why it works. A wrong candidate that differs from the truth in k rows changes a random batch's sum by about ±√k/2 (a random sum of k terms of ±1). If that exceeds 2E, the batch contradicts the candidate, by the same two-sided argument as the 4αn proof. So candidates with more than about $16E^2$ wrong bits get caught fast. Only about E^2 wrong bits can hide, which is small as long as $E^2 \ll n$, that is $E \ll \sqrt{n}$.

WATCH FOR

- The blue curve: the chance one random batch exposes a candidate with k wrong bits. It is near 0 for small k and rises to 1.
- The green curve: all m batches together. It rises earlier.
- As E grows the curves slide right: more wrong bits can hide.

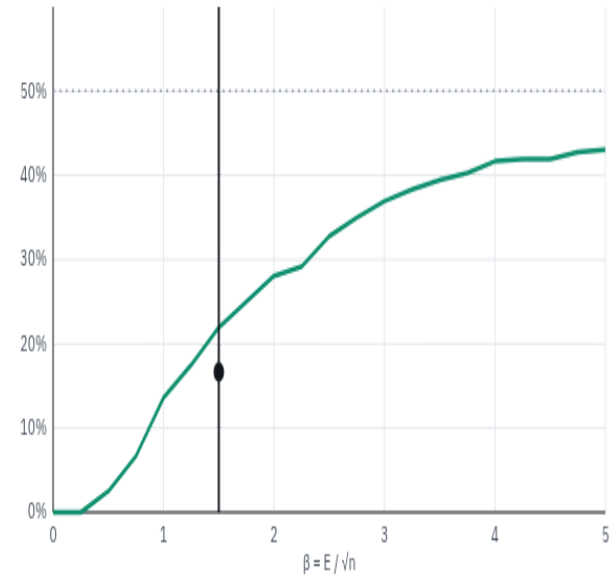
Recovered column

20 of 120 bits wrong (16.7%)

1	0	1	1✗	1	0✗	1	0	1	1	1	0	1	0	1
0	0	1	1	1	1	0✗	1✗	1✗	0	1	0	0✗	1	0
0	0	0	0	1✗	1	0✗	1	1	0	1	0	0✗	1	1
1	0✗	1	0✗	0	1	1	1	1✗	1	0	0	0	1	1
1	0	0	0	1	1	0	1✗	1	1	0	1	1	1	0
1	0✗	0	1✗	0	1	1	1	1	1	0	0	0	1	1
0	0✗	1✗	1	1	0	1	1	1	1	0	0	0	1	1
0	0	1✗	0	1	1	1	1✗	0	0	0	0	0	0	0✗

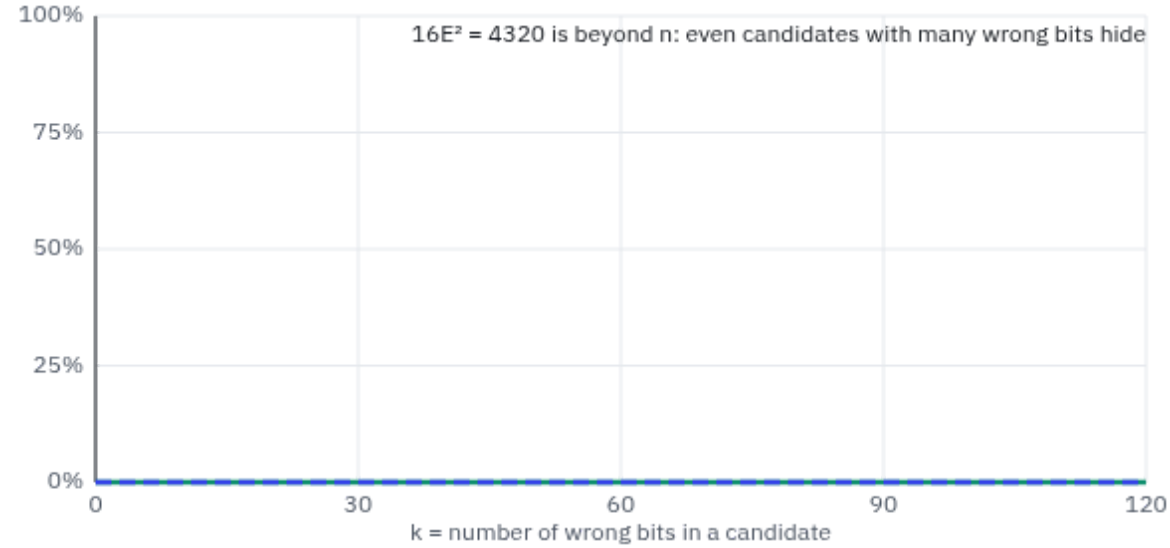
recovered correctly wrong bit

Error rate against noise scale β = E / √n



error (average of 3 runs) your current β
coin flip (50%)

Why random batches expose a wrong candidate the idea behind the proof



one random batch exposes a candidate with k wrong bits

all m batches together expose it

k = 16E²: below this, a wrong candidate usually hides

Watch the √n attack fail as noise grows

Now the full attack on a real run: n = 120 patients, m = 6n random batches, Gaussian noise. The noise scale is β = E / √n.

At β = 0 the column is recovered perfectly. As β grows the error rate climbs toward 50%, which is a coin flip. The transition happens around β of 1 to 3, that is, when E is of the same order as √n.

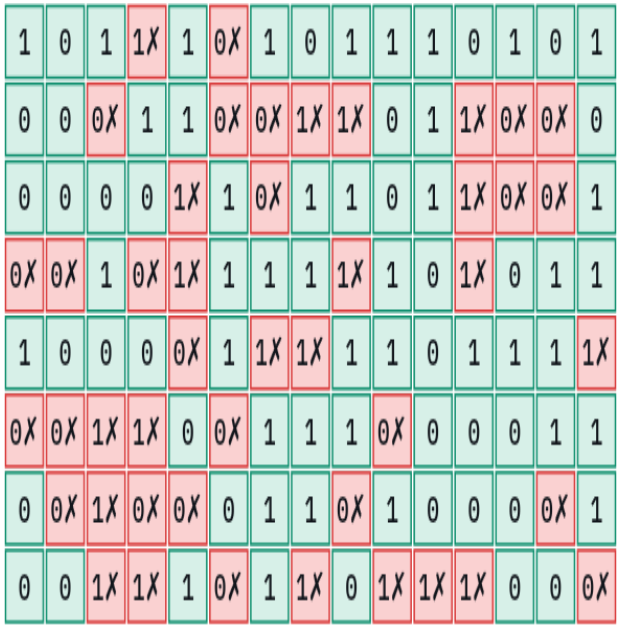
This is why privacy needs noise around √n or more: below it, even a cheap attack wins.

WATCH FOR

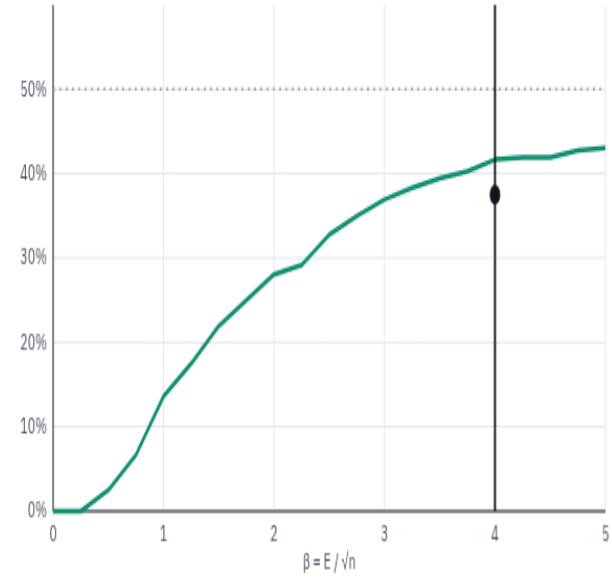
- The black dot slides along the error curve while the grid on the left fills with red errors.
- Try more queries (k) in the settings: it helps the attacker only a little, because the tolerated noise grows like √m.

Recovered column

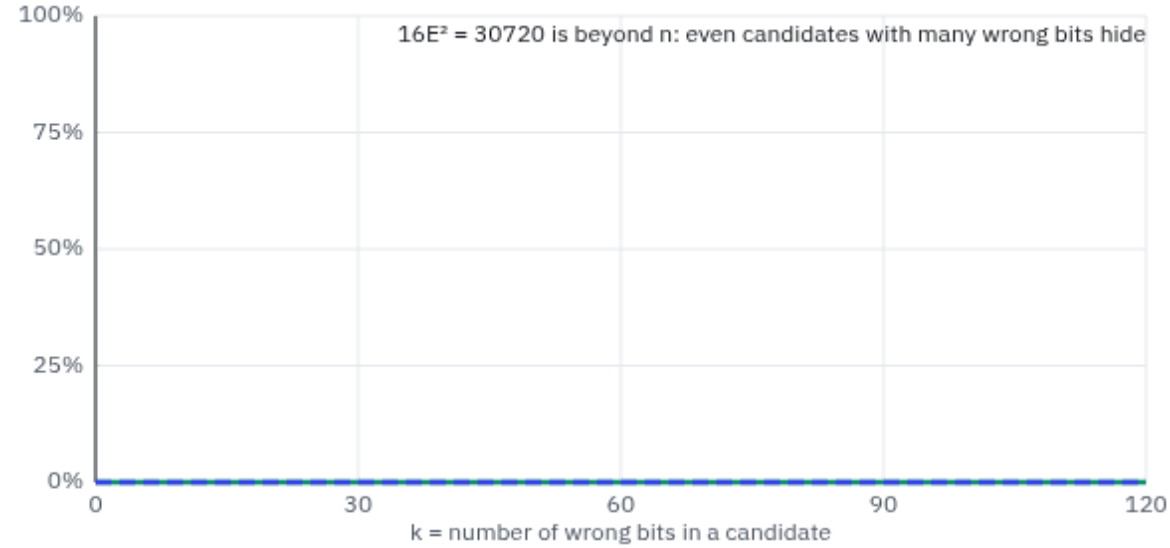
45 of 120 bits wrong (37.5%)



Error rate against noise scale β = E / √n



Why random batches expose a wrong candidate the idea behind the proof



one random batch exposes a candidate with k wrong bits

all m batches together expose it

k = 16E²: below this, a wrong candidate usually hides

CHAPTER 10 OF 10

What to take away

You've seen the full story: an attacker who only asks noisy group questions can still rebuild the secret column.

The cards on the right sum up the rules. When you're ready, open the **free lab** to change any setting yourself.

Open the free lab

Your lecture notes, mapped to this page

THE LECTURE SAYS	WHERE TO SEE IT
A query can be one record, the whole table, or any batch in between	Chapter 1. Click rows and ask your own batch.
Each answer is noisy, with error at most αn	Chapters 1–2. The blue window diagram.
Exponential attack: ask all 2^n queries, keep the consistent databases	Chapter 3.
If answers are within αn, reconstruction is within 4αn	Chapters 4–5 (proof on real data), chapter 6 (sweep over α).
Different noise distributions	Chapter 7.
Polynomial attack: works if the noise is below $\sqrt{n}$	Chapters 8–9.

Cheat sheet: the two theorems

Setting. Secret column $d \in \{0,1\}^n$. A query is any subset $S \subseteq \{1..n\}$ (one record, everyone, or anything between). The answer is

$$a(S) = \Sigma_{i \in S}\, d_i + e(S), \text{ with } |e(S)| \le E$$

Theorem 1 (exponential attack, $E = \alpha n$). Ask all 2^n subsets. Output *any* $c \in \{0,1\}^n$ with $|\Sigma_{i \in S}\, c_i - a(S)| \le E$ for every S. Then c differs from d in at most $4E = 4\alpha n$ positions.

Proof. d is consistent, so a consistent c exists. Let $A = \{i : c_i=1, d_i=0\}$ and $B = \{i : c_i=0, d_i=1\}$. On $S = A$, the count of d is 0 and the count of c is $|A|$, and both are within E of $a(A)$. So $|A| \le 2E$. The same argument on $S = B$ gives $|B| \le 2E$. Hamming$(c, d) = |A| + |B| \le 4E$. ∎ *Cost: 2^n queries and about $2^n \cdot 2^n$ checks.*

Theorem 2 (polynomial attack, Dinur–Nissim). If $E = o(\sqrt{n})$, then about $n \cdot \text{polylog}(n)$ *random* queries let a polynomial-time attacker recover all but $o(n)$ of the bits.

1. pick m random subsets $S_1 \ldots S_m$ (each i joins with probability ½)
2. find $c \in [0,1]^n$ with $|\Sigma_{i \in S_j}\, c_i - a(S_j)| \le E$ for all j (a linear program)
3. round: $c_i \ge ½ \to 1$, otherwise 0

Why. A candidate that differs from d in k positions changes a random subset's sum by about $\pm\sqrt{k}/2$. If that exceeds $2E$, the same two-sided argument as in Theorem 1 shows the batch rules the candidate out. So only candidates with $k \lesssim E^2$ wrong bits survive: few if $E^2 \ll n$. *The page demo uses least squares instead of the LP; exact constants and log factors are in the paper, so check them against your lecture notes.*

What the attack needs

Only the noisy answers. Not names, not ZIP codes, not any identifier. Hiding or coarsening identifiers (as k-anonymity does) does nothing against it.

The rule of thumb

- Noise of size αn: any survivor is at most $4\alpha n$ bits wrong.
- Noise well below $\sqrt{n}$: a fast attack with about n queries recovers almost everything.
- Noise about $\sqrt{n}$ or larger: both attacks stop working.

Why random noise isn't the full story

Independent noise averages out when you ask thousands of questions. A safe design must either cap how many questions get answered, or add enough noise that the total stays safe. That trade-off is what differential privacy formalises with its privacy budget ε.

Sources and caveats

This is the Dinur–Nissim reconstruction result. The 4αn proof, the exponential attack and the √n threshold are from that line of work. The "decoy" noise is my own simple construction to show the bound is not empty; it is not the only worst case.